Service Discovery Pattern

Category: Communication **One-liner:** A registry that lets services find each other by *name* at runtime, instead of hardcoding network addresses that change constantly.

(A **registry** is a central list of what exists and where; **runtime** = while the program runs; **hardcoding** = writing a value into source code so it can't change without editing and redeploying.)

Part 0 — Concepts you need first

- **IP address and port.** An **IP address** identifies a machine (`10.0.3.47`); a **port** identifies a program on it (`8080`). Together — `10.0.3.47:8080` — they identify one running service instance. To call a service you must know them — and in modern deployments **those numbers change all the time**. That is the whole problem.
- **Instance.** One running copy of a service. Five copies sharing load = five **instances**, each with its own IP. Adding copies rather than enlarging one is **horizontal scaling**.
- **Container.** An app packaged with everything it needs, starting anywhere in seconds; **Kubernetes** runs containers across a **cluster** of machines (§ Kubernetes basics). Containers are **disposable** — started, stopped, moved and replaced constantly, and **each new one typically gets a new IP**. An address looked up ten minutes ago may point at nothing.
- **Auto-scaling.** Automatically adding instances as load rises and removing them as it falls (e.g. **AWS** auto-scaling groups). It saves money — and means instance counts and addresses change all day with nobody touching anything.
- **Load balancing.** Spreading requests across instances so none is overwhelmed (§ Load balancers).
- **DNS.** The **Domain Name System** turns names into IP addresses (`example.com` → an IP). It's the internet's original name lookup, and some discovery systems (notably Kubernetes) extend exactly this idea inside the cluster.

Part 1 — The problem

In a **monolith**, component A calling B is a **method call in the same process** — no network, nothing to discover. In **microservices**, services are **loosely coupled** (each knows little of the others' internals), deployed independently, possibly on different continents; every call is a network call to an address that must be known. Three forces make hardcoded addresses unworkable:

1. **Dynamic infrastructure. Virtual machines** (software-simulated computers sharing a physical server), containers and Kubernetes give services **no fixed IPs or hosts**. An AWS auto-scaling group launches and terminates instances by load, **each time with new IPs**.
2. **Undetermined number of instances.** Which instance should a client call? Hardcode one and all traffic goes there — no load balancing, and if it dies everything fails. Maintaining a list by hand is error-prone and untenable at scale.
3. **Coupling.** If every client must know every service's address you have **tight coupling**: a location change means updating **configuration** in **every client that calls it**. Deployment details leak into clients and the independence you adopted microservices for disappears.

Concrete scenario: an Order Service must call Inventory (availability) then Payment. How does it find them? What happens when Inventory moves server, or extra Payment instances spin up under load — how does Order learn they exist?

Part 2 — The solution

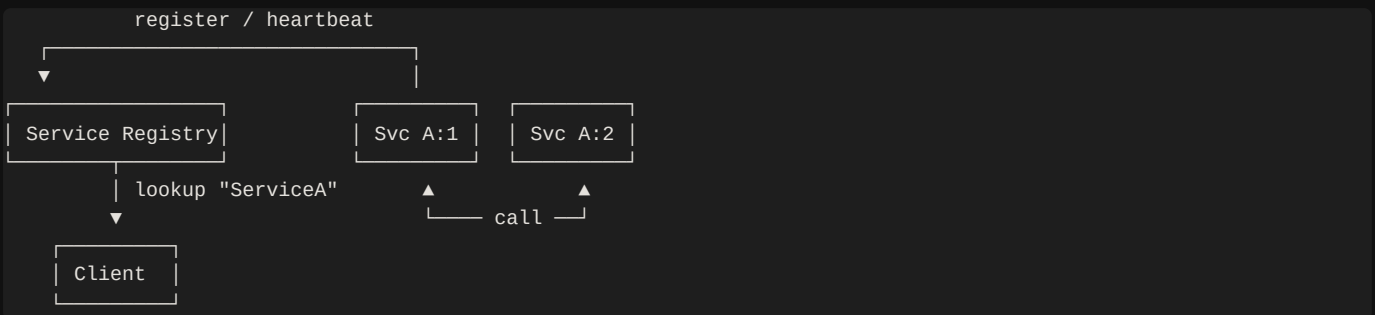
Introduce a **central registry and lookup mechanism**.

- **Dynamic registration** — each instance **registers its host and port with a Service Registry when it starts** and deregisters on shutdown, so the registry always lists active instances.
- **Lookup by logical name** — instances register under a name (`OrderService`, `UserService`); clients ask the registry for `"OrderService"` and get back the current location(s).

> **This indirection is the entire trick.** Addresses can change freely because clients only ever know the *name* — a phone book: you look someone up by name, and it doesn't matter that their number changed.

- **Load balancing and scaling** — discovery combines with a load-balancing strategy; scaling out is just **registering more instances, which clients discover automatically**.
- **Decoupled configuration** — clients hold no endpoint configuration; they supply a name and **routing happens at runtime**, which is what makes continuous deployment and auto-scaling possible without disrupting clients.

Part 3 — Architecture



The Service Registry

A database of all services and their current locations — the **centralized source of truth**. Three responsibilities:

1. **Registration** — on startup a service registers name, IP, port, and optional **metadata** (version, **health endpoint** — a URL answering "am I working?").
2. **Deregistration** — on graceful shutdown, or when the registry detects the instance is gone.
3. **Health checking** — the registry periodically **pings** services or their health endpoints; no response within a timeframe → failed and removed.

> *Why health checking is essential:* deregistration only covers graceful shutdown. An instance that **crashes**, loses power or is **partitioned** off the network (cut off by a network failure) never says goodbye. Without health checks the registry hands out dead addresses forever.

The payoff: a service moves or scales → the registry updates → others still find it; a service dies → removed → callers apply **fallbacks**.

Part 4 — Client-side vs. server-side discovery

Both use a registry; they differ in **who performs the lookup and load balancing** — the single most likely interview question here.

Client-side discovery — "smart client"

1. **Registration** — instances register at startup, perhaps with a **heartbeat** (periodic "still alive" message), and unregister on shutdown.

2. **Lookup** — the client **queries the registry directly** for active instances.
3. **Client-side load balancing** — the client (or a library) picks one, often **round-robin** (take turns) or **consistent hashing** (same key → same instance; adding/removing instances moves few keys — § Consistent hashing).
4. **Invocation** — the client calls **the chosen instance directly**; later requests repeat the lookup or use a **cached** result.

Classic stack: Netflix **Eureka** (open-source registry) + a client load-balancer library — **Ribbon** or **Spring Cloud LoadBalancer**.

Advantages: no extra hop after discovery; **intelligent, application-specific load balancing** (e.g. by request content) because the client sees all instances; **fewer moving parts** (no routing layer besides the registry); slightly lower latency (no intermediate **proxy**).

Disadvantages: every client needs the discovery library; **painful in polyglot environments** (many languages → a compatible client for each, a real barrier); **two network calls per request** (registry, then service — though caching usually removes the first); **couples clients to the discovery mechanism**, so changing the registry means updating many clients.

Standard mitigation: clients **cache** registry responses — no lookup per call, *and* the system keeps working if the registry is briefly down.

Server-side discovery — "smart router, dumb client"

1. **Registration** — identical.
2. **Request to load balancer** — the client calls a **known, fixed endpoint**: load balancer, gateway, DNS name, or **virtual IP** (a stable IP not tied to one machine). **The client never queries the registry.**
3. **Discovery & routing** — the load balancer queries the registry for healthy instances,
4. **picks one** with its own algorithm,
5. **forwards** the request. The client just called one endpoint.

Advantages: much simpler clients (no discovery code or libraries); **language-agnostic** — any client that can make a normal HTTP request works, the decisive advantage in a polyglot estate; **one network call per request**; **abstraction** — locations can change without affecting clients; **often provided out of the box** by Kubernetes and cloud load balancers.

Disadvantages: an extra hop; the **load balancer must be highly available and scalable** (critical to every request); **operational overhead** if your platform doesn't provide one; **the client loses routing control** and must trust the router.

Choosing between them

	Client-side	Server-side
Who queries the registry	The client	The load balancer / gateway
Network hops	Fewer (direct call)	One more (through the router)
Client complexity	Higher — needs a library	Very low
Polyglot friendliness	Poor	Excellent
Routing control	Fine-grained, app-specific	Whatever the router does
Typical stack	Eureka + Ribbon (Spring Cloud)	Kubernetes Services, cloud LBs

Practical guidance:

- In **Spring Cloud / Netflix OSS** (Java microservice libraries / Netflix's open-source tooling), client-side Eureka + Ribbon is straightforward.
- On **Kubernetes**, built-in DNS discovery means you lean server-side — just call `http://my-service`.
- **Not mutually exclusive** — client-side for internal calls plus an API gateway (server-side) for external traffic.

In modern systems — Kubernetes, **service meshes** (a proxy beside every service handling inter-service traffic) — **server-side discovery is the norm**, because the platform hides discovery behind DNS or a gateway.

Part 5 — Inner workings

Self-registration vs. third-party registration

	Self-registration	Third-party registration
Who registers	The service itself, on start/stop	A **service manager** that monitors services and updates the registry
Upside	Simple; service has full control	Burden removed from services
Downside	Service must keep its status current	Another component to manage

Health checks — two models

- **Self-reporting (heartbeat)** — services periodically tell the registry they're alive; no signal within the window → removed.
- **Active monitoring** — the registry pings services; no response → offline.

> Under load: heartbeats put work on the services and scale with instance count; active monitoring puts work on the registry and scales with *its* capacity to poll everyone.

Part 6 — Worked example (simplified Java)

The registry — a phone book

```java
import java.util.concurrent.ConcurrentHashMap;

public class ServiceRegistry {
    private ConcurrentHashMap<String, String> services = new ConcurrentHashMap<>();

    public void register(String serviceName, String serviceAddress) {
        services.put(serviceName, serviceAddress);
    }

    public String discover(String serviceName) {
        return services.get(serviceName);
    }
}
```

Keys are names, values addresses. **Why `ConcurrentHashMap`?** Many **threads** register and look up simultaneously; a plain `HashMap` corrupts under concurrent modification, `ConcurrentHashMap` is built for it.

A service registering itself, and a client discovering

```
public class Service {
    private String name;
    private String address;

    public void register(ServiceRegistry registry) {
        registry.register(name, address);      // self-registration at startup
    }
}

public class ClientService {
    private ServiceRegistry registry;

    public void callService(String serviceName) {
        String serviceAddress = registry.discover(serviceName);

        if (serviceAddress == null) {
            throw new RuntimeException("Service not found: " + serviceName);
        }

        // Make a request to the service
    }
}
```

In a real system registration is automatic — via a service manager or the deployment process.

What this toy version is missing

- **Health checks** to keep registrations current
- **A distributed registry** for traffic and **redundancy**
- **Failure handling in** `callService` — retry, or fall back to a backup service
- **A *list* of instances, not one address** — `Map<String, String>` stores one address per name, so this design **cannot load balance at all**. A real registry stores `Map<String, List<String>>` and returns all healthy instances.

That last gap is the difference between a demo and something usable.

Part 7 — Performance implications and special considerations

1. Latency overhead

Every lookup is an extra step, usually small: client-side discovery **caches** the instance list and refreshes every ~30 s or on a **cache miss**; a lookup is a millisecond-scale local call. To minimise it:

- Cache discovery results briefly rather than querying per request
- Keep the registry fast (in-memory **indexes**)
- Use **asynchronous** updates — pushes or **long-polling** (server holds the request open until there's news)
- In high-performance cases, bake a local endpoint cache into each instance at startup

2. Scalability of the registry

Thousands of registrations plus frequent lookups can make it a **bottleneck**.

- **Clustering** — several registry **nodes** (e.g. three Eureka or **Consul** — HashiCorp's registry/config tool — instances) sharing load; give clients multiple registry URLs
- **Sharding** by service domain — uncommon; one cluster usually suffices for a large organisation
- **Efficient heartbeats** — thousands every 30 s is real load; Eureka lets you tune the interval and has **self-preservation mode** to avoid overwhelming the network during an outage or partition
- **Lightweight protocols** — Consul uses **gossip** (nodes swap news with a few random peers, spreading it like a rumour); **Zookeeper** (Apache's coordination service) uses an event-driven model over

persistent connections

3. Fault tolerance and high availability

If the registry is down, services may not find each other — especially with client-side discovery. (§ Fault tolerance.)

- **Run multiple registry instances** (two or three in production), synced via **consensus** (nodes agree on one value despite failures — Consul, Zookeeper) or **peer-to-peer** copying (Eureka)
- **Client-side caching of last-known-good data** — clients keep using the last good instance list if the registry is unreachable; Spring's Eureka client does this by default. A registry outage then only blocks *new* registrations: **running services keep working** as long as topology hasn't changed drastically. *(The same trick protects config servers — Config Externalization.)*
- **Failover** — give clients several registry addresses so they try the next when one fails
- **Monitor the registry itself**, not just what it tracks
- **Persistence** — Consul, **etcd** (Kubernetes' distributed key-value store) and Zookeeper persist to disk and recover after restart; Eureka keeps state in memory and relies on clients re-registering. Persistence helps if every node reboots, but may be overkill if services re-register on startup

Consistency vs. availability — the CAP trade-off

During a network partition a distributed system must choose **consistency** (never serve wrong data, even if that means serving nothing) or **availability** (always answer, possibly stale) — § CAP.

	Zookeeper / Consul	Eureka
Choice	**CP** — consistency	**AP** — availability
Behaviour	Avoids serving possibly-stale data; may become unavailable until consensus is restored	Continues serving whatever it has, even if stale
On partition	May refuse to answer	**Self-preservation** — won't purge entries, so some info stays available

Know and tune your registry's behaviour — e.g. turn self-preservation **off** in small setups for quicker removal of dead instances.

> *Which is right?* For discovery, **availability usually wins**: a slightly stale list is survivable (try an address, it fails, retry another); a registry that refuses to answer means nothing can talk to anything. That's why Eureka chose AP.

4. Load balancing strategies

- **Round robin** — next instance in turn; even if instances have similar capacity
- **Random** — roughly balances over many requests; trivially simple
- **Least connections / load** — send to the instance with fewest active requests; balances *actual* load but needs tracking, so usually in server-side LBs or meshes
- **Hashing (sticky sessions)** — same client → same instance, for caching or **session affinity** (e.g. hash(user ID) mod instance count)
- **Geographic / AZ affinity** — prefer an instance in the client's **availability zone** (separate data centre within a cloud region) for latency; discovery metadata carries the zone

Ribbon defaults to round-robin; an AWS **ELB** does round-robin or least-connections by configuration.

5. Failover handling

With a discovered pool, a failed call can go to another instance:

- **Client retry** — on timeout or error, pick another instance (from the registry or cache) and retry; Ribbon and Spring Cloud LoadBalancer support this (Retry)
- **Circuit breakers** — **Hystrix** (Netflix, older) or **Resilience4j** (its modern Java replacement) stop calling a repeatedly failing target for a cooldown; with discovery this can be **per-instance**: stop

sending to failing instance X while Y and Z continue *(Circuit Breaker)*

- **Health-based removal** — the registry promptly marks failing instances unhealthy so lookups stop returning them — proactive, better than reacting per request
- **Short timeouts** — **fail fast** and try another instance, viable precisely because you have a pool

6. Consistency of data

The registry lags reality: a crashed instance may be handed out until the next heartbeat period. Mitigations:

- **Shorter heartbeat intervals** — faster detection, more chatter
- **Client pre-flight health check** before critical requests — overhead, rarely done
- **Tolerate the failed attempt and retry another instance** — usually the right answer
- Strongly consistent systems like Zookeeper notify watchers almost immediately on disconnect, but under high **churn** (instances constantly starting and stopping) an AP registry like Eureka may give better overall availability

7. Data volume and registry size

Hundreds of services and thousands of instances make a large dataset: registries **index by service name**; provision enough RAM/CPU. **Tagging and filtering** (e.g. only instances tagged `version: v2`) helps during **rolling deployments** (replacing instances a few at a time).

8. Transactional consistency — subtle but important

If a service registers **before it's ready**, clients call it while it's still warming up — loading caches, opening **connection pools** (reusable database/network connections), **JIT-compiling** hot code.

> **Best practice:** register only once fully initialised, and **deregister immediately when shutting down, before actually going offline.** A very common source of confusing intermittent errors right after deployments.

Part 8 — Security considerations

The registry knows where everything is and steers traffic; compromise it and an attacker can misroute traffic or map your topology. Treat it as **security-sensitive critical infrastructure**.

Threat	Mitigation
Service spoofing — a malicious instance registers as a legitimate service and **intercepts or manipulates data**	Authenticate and authorize registration: Consul **ACLs and tokens** (access-control lists checked against a secret token); Eureka with **HTTP basic auth or mutual TLS** (**mTLS**: both sides present certificates proving identity)
Unauthorized registry access — free querying gives an attacker **a map of all your services**	Internal network only (never expose Eureka publicly); authenticate API and **UI**; Consul ACLs for reads; Kubernetes DNS is cluster-isolated and **RBAC** (role-based access control) limits who reads what
Communication security — registry traffic may cross untrusted networks	**TLS** (the encryption behind HTTPS) on all registry traffic, especially multi-datacenter; with server-side discovery, secure client→gateway and gateway→service too
Least privilege — not every service needs to see every other	Segment the registry or use ACL policies; a "frontend" service sees `accounts` and `orders`, not internals
Audit and monitoring	**Audit trails**; alert on registrations from unknown IPs or spikes in failed auth
DDoS and abuse (floods of traffic from many machines)	**Capacity planning** and **rate limiting** on discovery operations
Server-side components accept client traffic *and* talk to the registry	Harden the gateway / load balancer accordingly

---

Part 9 — Real-world implementations

- **Netflix — Eureka.** Hundreds of services register with Eureka and look each other up, letting Netflix scale and move services across data centres without clients knowing. Proven at streaming scale.
- **Kubernetes — the most widespread use today.** A **Service** is a stable virtual IP and DNS name for a set of **pods** (Kubernetes' smallest unit: one or more containers). As pods come and go Kubernetes updates the endpoints list, and **kube-proxy** (a network component on every node) load-balances Service traffic to current pods. Any pod calls `http://service-x`. Under the hood: an **etcd**-backed registry plus cluster DNS (**CoreDNS**).

  > This is why "we're on Kubernetes" is usually a complete answer to discovery — building a second layer on top is usually a mistake.

- **Airbnb — SmartStack.** Pre-Kubernetes: **HAProxy** (open-source load balancer) on every host plus **ZooKeeper** as registry. Services registered via a local **agent**, and the local HAProxy reconfigured itself — every app got a local load balancer for other services. Built because manually tracking service pools had become untenable.
- **Consul (Stripe, PagerDuty).** DNS-based discovery plus an HTTP API; multi-datacenter support makes it popular where Kubernetes isn't the **orchestrator** or a platform-agnostic solution is needed.
- **Service meshes — Lyft's Envoy. Envoy sidecars** (a high-performance proxy deployed next to each service) **receive updates from a central control plane** about every service's location. Service A calls its local Envoy, which knows all instances of B and routes intelligently. **Discovery leaves application code entirely and moves into infrastructure** *(Sidecar)*.

Use cases

- **Microservices architectures** — inter-service connections become unmanageable at scale without it
- **Cloud-based systems** — services are placed dynamically by demand and cost and re-register wherever they land

Design example 1 — e-commerce

User, Inventory, Shopping Cart, Order. Without discovery, Inventory moving server forces User and Cart to update their connections; with it, Inventory re-registers and the others just query the registry.

Design example 2 — streaming platform

Profiles, video streaming, payments, recommendations. Streaming scales hard at peak, adding and removing instances; without discovery, callers hit instances that no longer exist; with it, instances register/deregister as they come and go and callers always get a valid one.

Part 10 — Common beginner misconceptions

- **"Service discovery replaces load balancing."** They're complementary: discovery says *what instances exist*; load balancing picks *which one*. Client-side puts both in the client, server-side both in the router.
- **"The registry is a single point of failure, so the pattern is bad."** It is a **SPOF**, which is why you cluster it *and* cache on clients. An outage should degrade you (stale data), not stop you.
- **"You need to install Consul or Eureka."** Not on Kubernetes — you already have DNS discovery. Another registry usually creates two competing sources of truth.
- **"Registering at startup is enough."** It must be paired with health checks and prompt deregistration, or dead entries pile up.
- **"DNS alone solves this."** DNS caching and **TTL** (how long a cached answer may be reused) were designed for rarely-changing addresses; clients and language **runtimes** (e.g. the JVM) cache DNS aggressively, sometimes forever. That's why Kubernetes pairs DNS with a stable virtual IP rather than returning raw pod addresses.

Part 11 — Check your understanding

1. Why doesn't a monolith need service discovery?
2. What is the "indirection" at the heart of the pattern?
3. Give the decisive advantage of server-side discovery in a polyglot organisation.
4. Why does Eureka choose availability over consistency?
5. Why must a service register only *after* it's fully initialised?
6. What's the most important gap in the toy Java registry example?

(Answers: 1 — components call each other in-process, there's no network address to find; 2 — clients know logical names, not addresses, so addresses can change freely; 3 — it's language-agnostic, since any client that can make an HTTP request works, with no per-language discovery library; 4 — a stale instance list is survivable via retry, but a registry that won't answer stops everything; 5 — otherwise clients discover and call it while it's still warming up, causing intermittent errors after deploys; 6 — it maps one name to one address, so it can't return multiple instances and therefore can't load balance.)

Part 12 — Interview soundbite (30 seconds)

> "Service discovery solves the fact that in a dynamic environment services have no fixed addresses — containers and auto-scaling groups get new IPs constantly, so hardcoding endpoints couples every client to every service's deployment. Instances register with a registry at startup under a logical name and deregister on shutdown, with health checks or heartbeats removing dead ones; clients look up by name. Two flavours. Client-side: the client queries the registry and load-balances itself — Eureka plus Ribbon — no extra hop and smart routing, but a discovery library in every language. Server-side: the client calls a fixed load balancer or gateway that does the

> lookup — simpler and language-agnostic, but an extra hop and the LB becomes critical. Kubernetes does server-side for you via CoreDNS and an etcd-backed registry, so on Kubernetes you usually shouldn't add another layer. Key considerations: run the registry as an HA cluster, cache on clients so a registry outage degrades rather than kills you, and the CAP trade-off — Consul and Zookeeper are CP, Eureka is AP and serves stale data rather than going down, usually the right call here. And secure it: anyone who can write to the registry can spoof a service and intercept traffic."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/ServiceDiscoveryDemo.java`

```
cd demos && ./gradlew runCh05
```

The most important line — the fix for the toy example's fatal flaw:

```
// A LIST of instances per name - Map<String,String> could never load balance
private final Map<String, Map<String, Registration>> services = new ConcurrentHashMap<>();
```

Eviction by heartbeat TTL — catches crashed instances that never deregistered:

```
void evictExpired() {
    long now = System.currentTimeMillis();
    services.forEach((name, m) -> m.entrySet().removeIf(e ->
        now - e.getValue().lastHeartbeat > ttlMs));
}
```

And the caching that makes a registry outage survivable:

```
List<Instance> instancesOf(String serviceName) {
    try {
        List<Instance> fresh = registry.discover(serviceName);
        cache.put(serviceName, fresh);
        return fresh;
    } catch (IllegalStateException e) {
        return cache.getOrDefault(serviceName, List.of());   // last known good
    }
}
```

B. What you should see

```
=== 2. Client-side discovery + round robin ===
  call 1 -> 10.0.0.2:8080
  call 2 -> 10.0.0.3:8080
  call 3 -> 10.0.0.1:8080      <- cycles evenly

=== 4. An instance dies (stops heartbeating) ===
    [registry] EVICTED 10.0.0.3:8080 (no heartbeat)
  >> crashed instance removed WITHOUT graceful deregistration

=== 5. The stale-entry window ===
  a client discovering NOW still gets: [..., 10.0.0.4:8080]
  >> unavoidable. The fix is not prevention but RETRY ON ANOTHER INSTANCE

=== 6. Registry outage - cache keeps the system alive ===
    [client] registry DOWN - using cached list (3 instances)
  still routing to 10.0.0.2:8080
  >> running services survive; only NEW registrations are blocked

=== 7. Filtering by metadata (canary routing) ===
  all instances: [10.0.0.2, 10.0.0.1, 10.0.0.4, 10.0.0.5]
  v2.0 only:     [10.0.0.5:8080]
```

Sections 5 and 6 teach the most. The stale-entry window is unavoidable — there's always a gap between death and detection — so the answer is **retry against another instance**, not better detection. Section 6: the registry is down and traffic still flows, thanks to client-side caching.

C. Production — you probably already have this

Kubernetes (do this by default). Discovery is built in:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service          # matches pods by label
  ports:
    - port: 80
      targetPort: 8080
```

Any pod calls `http://order-service`; CoreDNS resolves it and kube-proxy balances across healthy pods. **Readiness probes are the health check** — the piece people skip. (A **probe** is a periodic check Kubernetes runs against a container; `/actuator/health` is Spring Boot Actuator's health URL.)

```yaml
readinessProbe:                 # "am I ready to receive traffic?"
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  initialDelaySeconds: 10
  periodSeconds: 5
livenessProbe:                  # "am I alive, or should I be restarted?"
  httpGet: { path: /actuator/health/liveness, port: 8080 }
```

> **Readiness enforces the Part 7.8 rule**: an unready pod is removed from the Service endpoints, so no traffic reaches it while warming up.

Spring Cloud + Eureka (client-side, when not on Kubernetes). A **lease** is a registration that expires unless renewed by heartbeats; `RestTemplate` is Spring's classic HTTP client; `@Bean` has Spring create and manage the object:

```yaml
eureka:
  client:
    serviceUrl.defaultZone: http://eureka-1:8761/eureka,http://eureka-2:8761/eureka  # failover
    registryFetchIntervalSeconds: 30      # cache refresh
  instance:
    leaseRenewalIntervalInSeconds: 10     # heartbeat frequency
    leaseExpirationDurationInSeconds: 30  # TTL before eviction
```

```java
@LoadBalanced                              // makes RestTemplate resolve service names
@Bean RestTemplate restTemplate() { return new RestTemplate(); }

// Call by LOGICAL NAME, not address
restTemplate.getForObject("http://order-service/orders/{id}", Order.class, id);
```

Graceful shutdown — deregister *before* going offline:

```yaml
server.shutdown: graceful
spring.lifecycle.timeout-per-shutdown-phase: 30s
eureka.client.shouldUnregisterOnShutdown: true
```

D. Implementation pitfalls

1. One address per service name. The toy example's flaw: name → single address is a naming service, not discovery — no load balancing, no failover.

2. Registering before you're ready. Register only after caches are warm and pools open, or clients get random-looking errors right after deploys. Gate traffic with a **readiness** probe, not a liveness probe.

3. No client-side caching. The registry becomes a hard dependency of every request — a SPOF that stops everything. With caching, an outage is an inconvenience.

4. Heartbeat TTL too long or too short. 90-second TTL = up to 90 seconds of traffic to a dead instance; too short and you evict healthy instances during a **GC pause** (the JVM briefly halting to reclaim memory) or network blip. 10s heartbeat / 30s expiry is a reasonable start.

5. Not deregistering on shutdown. Every deploy leaves a window of **connection-refused** errors (nothing listening on that port) from already-stopped instances.

6. Adding a registry when you're on Kubernetes. Eureka inside Kubernetes gives two sources of truth that disagree during rollouts. Use the platform.

7. Treating discovery as a substitute for retry. Discovery tells you what *was* alive. Always pair it with retry-on-another-instance *(Retry)*.

8. Leaving the registry unauthenticated. Anyone who can write to it can register a malicious instance under a real service name and intercept traffic. Enable ACLs or mTLS.

Related patterns

- **API Gateway** — typically the server-side discovery router
- **Circuit Breaker** — per-instance breakers over a discovered pool
- **Retry** — retry against a *different* discovered instance
- **Sidecar** — service mesh sidecars (Envoy) absorb discovery entirely
- **Config Externalization** — services need discovery to find the config server too